Final Assignment

In this assignment, each student, working in a pair, will practice concepts learned on the second half of this course such as *aggregations*, *strings*, *containers*, File IO operations, *error handling*, *algorithms*, *lambda expressions*, and *function pointers*. To accomplish this, each pair of students will create a library called **library**[1] that will handle information about students, courses, and programs a Seneca.

General Instructions

For this assignment, you will work with the provided source code **main.cpp**. Note that this file should not, under no circumstances, be changed. You need to create your module in such a way that it works properly with the provided **main()** function as it is.

Note that this assignment differs from regular labs in significant ways:

- Its size is significantly larger than that of a regular lab.
- Instructions are "vaguer". I.e., I omit the details of how to implement each class and leave them to the discretion of each pair of students.
- Each pair of the students will have to "present" their code to the instructor. I.e., during lab hours, the pair of students must attend the lab section and be ready to answer questions from the instructor about their code.
- Collaboration between students outside of the pair is strictly forbidden. Any signs of collaboration between different teams will be considered an academic honesty offense.
- Relying heavily on AI to write your code will also be considered an academic offense. If students cannot explain the code they submitted during the interview, it will be treated as a form of plagiarism. Using AI for brainstorming ideas or verifying the correctness of code is acceptable. However, the final code must ultimately be the result of the students' own intellectual work.

This assignment will require each group to create a module called **library**. This module should contain a header file, **library.h**, containing all classes declarations, and an implementation file, **library.cpp**, containing definitions of classes member functions. Make sure to add preprocessor directives (such as **#ifndef**, **#define**, etc.) to ensure that there is no risk of *double inclusion* of header files.

Each pair of students must enroll in a group using Blackboard's groups tool, as presented by your instructor in class. For submission, each pair of students should submit their **library.cpp**, **library.h**, and **main.cpp** files on Blackboard prior to the due date. A deduction of 20% per day will be applied to late submissions. Make sure to submit the files directly. Do not place the files in a folder and zip it for submission.

[1] I apologize for the uninspired name!

Specific Instructions

You should create a module called **library** with the specifications below:

- A const int **COURSE_SIZE** with value **6**[2]
- An enumeration called **gender** that should contain three possible values: **male**, **female**, and **other**

An abstract class called **Person** which should contain the following members:

- A protected **string** to store a full name
- A protected **string** to store a date of birth
- A protected **gender** (see enumeration above), to store the person's gender
- A public constructor that takes no arguments and sets all the **Person**'s member variables default values. Feel free to choose your default values.
- A public constructor that takes three arguments of types **string**, **string**, and **gender**. These arguments are used to set the class' three protected member variables.
- A pure virtual destructor
- A virtual function called **display** that takes no arguments and returns void. When called this function prints all info about the Person's member variables
- A pure virtual function called **name** that takes no arguments and returns a **string**. This function should work as a getter for the full name of the **Person**.

A class called **Student**, derived from **Person**, with the following members:

- A private vector of type pointer to **Course** (see next class) that stores the courses in which the student is enrolled at
- A private double to store the student's GPA
- A public constructor that takes no arguments and sets all the **Person**'s member variables default values. Feel free to choose your default values
- A public constructor that takes four arguments of types **string**, **string**, **gender**, and **double**. These arguments are used to set the class' member variables
- A public copy constructor that copies all member variables from one object to another
- A public destructor that simply prints that the student object has been destroyed
- A public function called display that takes no arguments and returns void. This function, when called, displays all info about a student, besides the courses in which the student is enrolled
- A public function called **name** that takes no arguments and returns a **string**. This function should work as a getter for the full name of the **Student**.
- A public function called **add_course** that takes as argument an object of type **Course** by reference. This function, when called, should add the address of the provided course to the student's **courses** vector. Also, this function should call the **add_student** function for the **Course** class (see below). This function, **add_course**, should be implemented with error handling in place. I.e., it should try to add the student to the course but, if the course if already full[3], it should handle exceptions that are thrown by the **add_student** function.

---

[2] This very small side was chosen to facilitate testing. In a real scenario, this size would be much higher.

[3] I.e., it already has a number of students equal to COURSE_SIZE

A class called **Course** with the following members:

- A private string to store the course's name (for example, *Mathematics for Developers*)
- A private string to store the course's code (for example, *BTL200*)
- A private vector of type pointer to **Student** (see previous class) that stores the memory addresses of the students that are enrolled in this course
- A public constructor that takes no arguments and sets all the **Course**'s member variables default values. Feel free to choose your default values.
- A public constructor that takes two arguments of type **string**. These arguments are used to set the course's name and code.
- A public destructor that simply prints that the course object has been destroyed
- A public function called **display** that takes no arguments and returns void. When called, this function should display the *name* and *code* of the course, as well as the *number of students* that are currently enrolled.
- A public function called **code** that takes no arguments and returns a **string**. This function should work as a getter for the **Course**'s *code*.
- A public function called **add_student** that takes as argument an object of type **Student** by reference and returns void. This function, when called, tries to add the address of the provided student to the *students*' vector. However, if there are already COURSE_SIZE students enrolled, it should throw an error message.
- A public function called **display_students()** that takes no arguments and returns void. This function, when called, prints the name of all students enrolled in this course. **Important:** Use **for_each** to loop through all enrolled students!

A class called **Program** with the following members:

- A private string to store the program's name (for example, BSA)
- A private vector of type pointer to **Student** that stores the memory addresses of the students that are enrolled in this program
- A private vector of type pointer to **Course** that stores the courses in this program
- A public constructor that takes an argument of type string and sets the Program's *name* accordingly.
- A public destructor that simply prints that the program object has been destroyed
- A public function called **add_student** that takes a Student object by reference and adds the address of this object to its *students* vector. This function returns void.
- A public function called **add_course** that takes a Course object by reference and adds the address of this reference to its *courses* vector. This function returns void.
- A public function called **honours_list()** that takes no arguments and returns void. This function, when called, should print a list of the names of all students who have a GPA superior to 3.5. This list should be printed in descending order. Note that you should make sure to use methods from the algorithm library to perform sorting, copying, and loops. *Hint: for this function to be able to access the student's GPA, you should make this entire Program class a friend of the class Student.*

- A public function called **save_to_file()** that takes as an argument a **const** reference to a **string** (or a **const char array**) and, as the name suggests, will save into a text file all the contents of the **Program**. I.e., it will save the information about all courses in the program, as well as information about all students in the program. An example of a saved program is provided on Blackboard. Note that your result does not need to be identical to the provided.
- A public function called **load_from_file()** that takes as an argument a **const** reference to a **string** (or a **const char array**) and, as the name suggests, will load the information from a text file into a new **Program** object. It is important to notice that this function needs to read the file in a way that perfectly matches the way in which the file was saved via **save_to_file()**. *Hint: If your file is saved so that multiple pieces of information gets stored in the same line, you might need to use **getline()**, instead of just the insertion operator.*

Evaluation

Source Code: 2.0 Marks

Your source code will be evaluated based on how much of the overall functionality it implements, as well as how well it adheres to proper coding design practices (see last page). Make sure to upload all three source code files on Blackboard.

Presentation: 8.0 marks

Each student will be asked four questions. Each question is worth up to two marks, according to the rubrics presented below:

Level	Description	Marks
Perfect Answer (No Hesitation)	The student answers immediately, confidently, and correctly. The explanation is clear, technically accurate, and demonstrates full understanding of the concept or code being discussed.	**2.0**
Correct Answer (Some Hesitation)	The student answers correctly but shows minor hesitation, pauses briefly to think, or needs a moment to organize the explanation. Understanding is still clearly demonstrated.	**1.5**
Correct Answer With Help / Significant Hesitation	The student eventually answers correctly but requires hints, guiding questions, or displays significant hesitation before reaching the correct explanation.	**1.0**
Partially Incorrect / Major Hesitation	The student shows some understanding but provides an incomplete or partially incorrect explanation. The answer may contain errors or demonstrate confusion about important aspects of the code.	**0.5**
Incorrect or No Answer	The student provides an incorrect explanation, is unable to answer the question, or declines to attempt an answer.	**0.0**

Proper Code Design

In what follows, I provide a guideline of what amounts to a proper coding style.

- Keep your code simple and focus on using the libraries and methods that we covered in class. Usage of libraries that were not covered will result in lost marks.
- Excerpts of code that are repeated in multiple parts of your source code should probably be encapsulated within a function. Then, this function should be called every time the excerpt of code is required.
- Call base class member functions from within derived class member functions whenever this can lead to less repetition of lines of code
- Use comments whenever you are doing something that would not be self-evident to someone reading your program. Do not use comments to explain what it is obvious from the code itself
- Use proper indentation.
- Do not violate the Single-Entry, Single-Output rule for functions and loops.
- Put blank lines above and below functions to separate them from each other.
- Don't put in extra blank lines. (Some people put a blank line between every line of code!)
- Read and understand the specifications. If you do not understand the specifications, ask me for clarification. If you do not implement something required, you will lose marks, even if you didn't understand the requirement - i.e. it is your job to seek clarification.
- Develop your code anywhere you like, but make sure your code runs under Visual Studio or g++. I will test your code using one of these two compilers.
- Use meaningful names for your variables, functions, and classes. For example, if you have a variable that stores a first name, it is better to call it **first_name** or **fname** instead of **var** or **x**.
- Do not create variables to hold values from the outputs of functions if these values are only used once. For example, given that the output of a function called **func1** is either 1 (for true) or 0 (for false), you can use it inside an if statement as:

  if (func1())
  instead of
  useless_var = func1()
  if(useless_var)